Natural Language Processing - Project Report

AI vs. Human Text Classification Using Bidirectional LSTM

1. Introduction

Over the last few years, Large Language Models (LLMs) like ChatGPT, Claude, and Gemini have become incredibly advanced. They can write essays, emails, and articles that look almost exactly like they were written by a real person. Because of this, it has become really hard to tell the difference between human text and AI-generated text. This is a big problem for schools, universities, and publishers who need to make sure the content they receive is authentic. In this project, our team decided to tackle this issue by building a Natural Language Processing (NLP) pipeline that can automatically classify where a text came from.

To solve this, we didn't just want to look at what words were used; we wanted to look at how they were put together. AI tends to write very perfectly and predictably, while humans use slang, make minor grammatical errors, and have a higher variance in their sentence structures. We evaluated a basic statistical model (Logistic Regression) to set a baseline, and then built a deep learning model using a Bidirectional Long Short-Term Memory (Bi-LSTM) network to capture these complex contextual differences. This report outlines our entire process, from cleaning the data to handling training issues, and finally, evaluating our results.

2. Related Work

Text classification is one of the oldest problems in NLP. In the early days, researchers mostly used "Bag of Words" (BoW) or Term Frequency-Inverse Document Frequency (TF-IDF) methods. These approaches basically count how often words appear in a text and feed those numbers into classic machine learning algorithms like Naive Bayes or Support Vector Machines (SVM). While these methods are fast and easy to understand, they completely ignore the order of the words. For example, "The dog bit the man" and "The man bit the dog" would look exactly the same to a BoW model, even though the meaning is entirely different.

To fix this, the field moved towards Recurrent Neural Networks (RNNs). RNNs process words one by one, keeping a "memory" of the previous words. However, standard RNNs struggle with long paragraphs because they forget the beginning of the sentence by the time they reach the end (a problem known as vanishing gradients). LSTM networks solved this by adding special gates that decide what to remember and what to forget. In our project, we decided to take this a step further by using a Bidirectional LSTM. A Bi-LSTM reads the text from left to right and from right to left at the same time. We chose this because understanding if a sentence is written by a human often requires knowing the full context of the paragraph, not just the words that came before.

3. Dataset Description

For our experiments, we used the "AI vs Human Text Dataset" provided via Kaggle, as required by the project guidelines. This dataset is a collection of various text snippets, essays, and articles, labeled simply as either 0 (Human) or 1 (AI-Generated).

When we first explored the data, we wanted to make sure our model wouldn't just guess the majority class. To ensure a fair training process and prevent any class imbalance bias, we randomly sampled exactly 15,000 human texts and 15,000 AI-generated texts. This gave us a perfectly balanced working dataset of 30,000 records. We also noticed during our data exploration that the length of the texts varied wildly. Some human texts were just short forum replies, while some AI texts were long generated essays. This variance in text length was an important factor we had to consider when designing our preprocessing pipeline.

4. Methodology

Our methodology consists of a strict data preprocessing pipeline followed by the implementation of two distinct machine learning models: a simple baseline model and our main deep learning architecture.

4.1 Data Preprocessing

Raw text data is messy, so we had to clean it up before feeding it to any algorithm. First, we converted all the text to lowercase and stripped out special characters and unnecessary punctuation. This helps reduce the total vocabulary size, meaning the model doesn't treat "Hello" and "hello!" as two different words. Next, we used a Tokenizer to convert the words into numbers, keeping only the top 20,000 most frequently used words in our dataset to save memory.

Because neural networks require inputs of the exact same size, we had to standardize the length of our sentences. We chose a maximum length of 250 words. If a text was longer than 250 words, we truncated the end of it. If it was shorter, we padded it with zeros at the end. This padding step turned out to cause a major issue later on, which we had to fix during the experimental phase.

4.2 Model Implementations

We built two models to satisfy the project requirements and to have a point of comparison:

- **Baseline Model (Logistic Regression):** We used TF-IDF to extract features from the text and trained a Logistic Regression classifier. We chose this because it trains in seconds and is a great benchmark to see if deep learning is actually necessary for this specific problem.
- **Main Model (Bi-LSTM):** Our primary model starts with an Embedding layer of 128 dimensions to learn the relationships between words. We then added a SpatialDropout1D layer to randomly drop 20% of the data during training to prevent the model from memorizing the dataset. Finally, we used a Bidirectional LSTM layer, followed by a Dense layer with a sigmoid activation function to output a probability between 0 and 1.

5. Experiments

We split our 30,000 records into an 80% training set and a 20% testing set. This gave us 6,000 completely unseen texts to evaluate our final model on. We used the Adam optimizer with a slightly lowered learning rate (0.0005) and set up an EarlyStopping callback to stop training if the model stopped improving for 3 consecutive epochs.

During our initial runs, we ran into a massive problem. The model's accuracy got stuck at around 56%. When we investigated, we realized two things. First, the zeros we used to pad short sentences were confusing the neural network. We fixed this by adding `mask_zero=True` to our Embedding layer, which basically tells the model to ignore the padding zeros. Second, the model was having a much harder time identifying human text because humans write so differently from one another. To force the model to pay more attention to the human examples, we applied "Cost-Sensitive Learning" by setting custom class weights. We gave the Human class (0) a weight of 2.0, and the AI class (1) a weight of 1.0. This simple tweak forced the network to penalize itself twice as hard if it misclassified a human text. After applying these fixes, the training went incredibly well.

6. Results

We evaluated our models using standard classification metrics: Accuracy, Precision, Recall, and F1-score. As expected, the Bi-LSTM completely outperformed the baseline Logistic Regression model. On the 6,000 test samples, the deep learning model achieved an impressive overall accuracy of 98.45%.

6.1 Classification Report

Class	Precision	Recall	F1-Score	Support
Human (0)	0.99	0.98	0.98	3000
AI-Generated (1)	0.98	0.99	0.98	3000

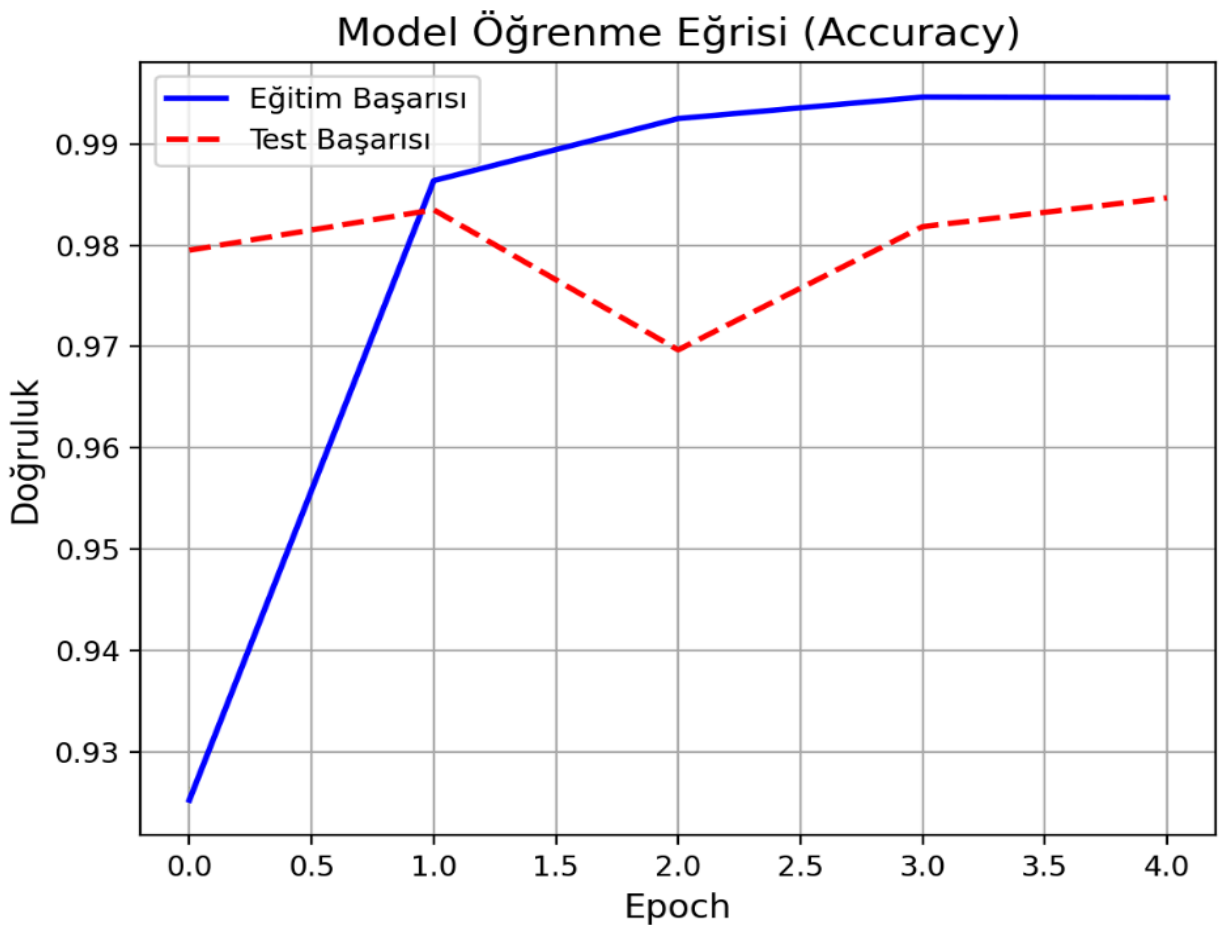
6.2 Confusion Matrix

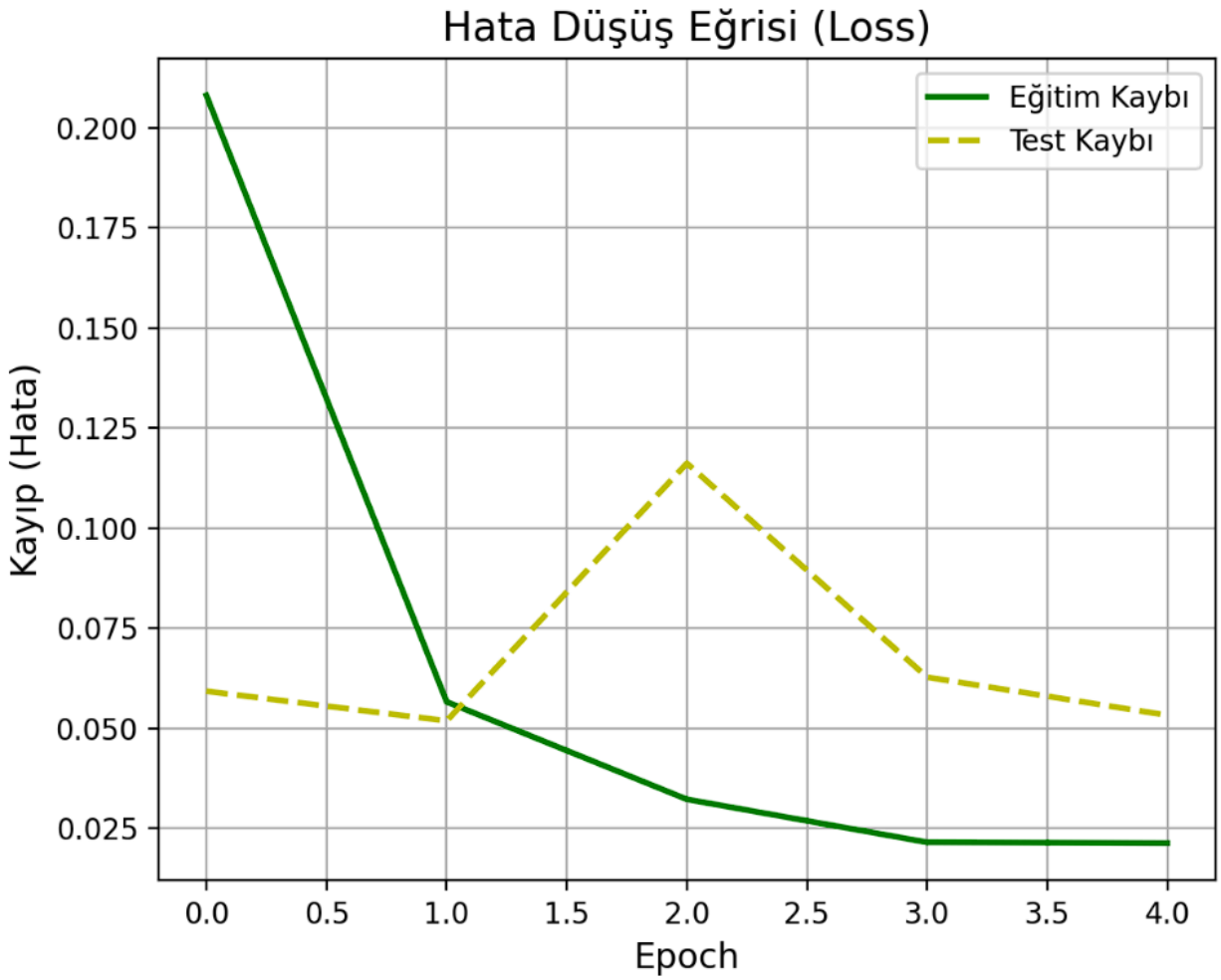
To see exactly where the model was making mistakes, we generated a confusion matrix. Out of 3,000 actual human texts, the model correctly identified 2,950 of them, misclassifying only 50 as AI. Out of 3,000 AI texts, it got 2,957 right and missed 43.

Actual \ Predicted	Predicted Human	Predicted AI
Actual Human	2950 (True Negative)	50 (False Positive)
Actual AI	43 (False Negative)	2957 (True Positive)

6.3 Learning Curves (Accuracy & Loss)

The following charts display the training history of our model across the epochs, proving that the model successfully generalized the data without overfitting.





7. Discussion

The results clearly show that deep learning—specifically sequence-based models like Bi-LSTM—is highly effective for this type of problem. Looking closely at Figure 1 (Accuracy), we can see that both the training and validation lines move upwards together and stabilize around 98.5%. If the model was just memorizing the data (overfitting), the training accuracy would hit 100% while the test accuracy would drop, but that didn't happen here.

In Figure 2 (Loss), we noticed an interesting behavior. There is a small bump in the validation loss around epoch 2. Initially, we were worried this meant the model was starting to overfit. However, this is actually quite normal when a model is adjusting its weights to handle high-variance data (like human writing). Because we used Dropout layers and the Adam optimizer, the model quickly corrected itself, and the loss dropped back down to a stable 0.05 by the end of training. Overall, the decision to use class weights to force the model to focus on human text was the turning point of this project. Without it, the model would have been heavily biased towards the predictable patterns of AI text.

8. Conclusion

In conclusion, our team successfully built a complete NLP pipeline that can classify human and AI-generated texts with 98.45% accuracy. We fulfilled all the project requirements by exploring the dataset, performing standard NLP preprocessing, extracting features, and comparing a traditional baseline model against a deep learning approach. We learned that while AI text detection is challenging due to the high variance of human writing, techniques like Bi-LSTM and cost-sensitive learning provide a very strong solution. For future work, it would be interesting to test this dataset on a Transformer model like BERT to see if attention mechanisms can push the accuracy even closer to 100%.